

DA6401W - Assignment 6

Department of Data Science and AI, IIT Madras

April 2026

Topics covered: CNN, observing learned filters and receptive fields, RNN, backpropagation through time, gating units, LSTM, GRU, Attention, multi-head attention.

Submit your solutions in one PDF file with all answers, codes, plots and outputs written/printed in that file.

1. (Programming: Convolution using NumPy)

Implement a 2D convolution operation from scratch using only NumPy (no deep learning libraries). Your implementation must support both **stride** and **padding**.

(a) Write a function:

```
def conv2d(image, kernel, stride=1, padding=0):  
    # your code
```

(b) The function should:

- Apply zero-padding to the input image
- Slide the kernel across the image using the given stride
- Compute element-wise multiplication and sum
- Return the output feature map

(c) Use the following inputs:

$$\text{Image } \mathbf{X} = \begin{bmatrix} 1 & 2 & 3 & 0 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 1 & 2 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 2 \\ 1 & 0 & 2 & 3 & 1 \end{bmatrix}$$

$$\text{Kernel } \mathbf{K} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

$$\text{Stride} = 2, \quad \text{Padding} = 1$$

(d) Print:

- The padded image
- The Output Feature Map

2. **(RNNs vs. LSTMs vs. GRUs)** Implement and compare recurrent architectures (RNN, LSTM and GRU) that utilize gating mechanisms to control information flow and mitigate the vanishing gradient problem.

- **Dataset:** IMDB Movie Review Dataset (Text Classification). Use a vocabulary size of 10,000 and pad/truncate sequences to a fixed length of 200 words. Its a binary classification problem (Positive vs Negative sentiment)
- **Architecture:** Build three separate models to compare:
 - – Embedding Layer → one hidden layer of Vanilla RNN → Dense (Sigmoid).
 - – Embedding Layer → one hidden layer of LSTM Layer → Dense (Sigmoid).
 - – Embedding Layer → one hidden layer of GRU Layer → Dense (Sigmoid).
- Train all model for 5 epoch, Adam optimizer, learning rate 0.001
- **Evaluation Metric:** Binary Cross-Entropy Loss, Accuracy, and F1-Score.

Dataset

```
from tensorflow.keras.datasets import imdb
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)
```

Observe and Report

- Plot the validation accuracy and validation loss for all three models on the same graph.
- **Report:** Compare the final performance and convergence speed. Explain how the specific gating units (Forget/Input/Output for LSTM; Reset/Update for GRU) allow these networks to outperform the Vanilla RNN on this dataset. Compare the training time per epoch between the LSTM and GRU.

3. **(Programming: Multi-Head Self-Attention using PyTorch)**

In transformer architectures, the attention mechanism allows the model to focus on different parts of the input sequence. Multi-head attention improves the model's ability to capture different representation subspaces.

The scaled dot-product attention is defined as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

where Q , K , and V represent query, key and value matrices respectively, and d_k is the dimension of the key vectors.

(a) Implement a PyTorch module for scaled dot-product attention.

```
class ScaledDotAttention(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, Q, K, V):
        # your code
```

(b) Implement a PyTorch module for multi-head self-attention.

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        # your code
```

(c) Your implementation must perform the following steps:

- Compute Query, Key and Value projections from the input.
- Split the embeddings into multiple heads.
- Apply scaled dot-product attention independently to each head.
- Concatenate the outputs of all heads.
- Apply a final linear projection layer.

(d) Use the following configuration:

$$d_{model} = 8, \quad h = 2, \quad \text{sequence length} = 4$$

Generate a random input tensor:

```
X = torch.randn(1,4,8)
```

(e) Print the following:

- Query, Key and Value matrices
- Attention score matrices
- Attention weight matrices
- Final output tensor

4. Implement a PyTorch module for **masked multi-head self-attention** (causal attention), where each position in the sequence can only attend to itself and previous positions.

Background: In transformer models, **self-attention** allows each token in a sequence to attend to all other tokens. In **masked** (causal) self-attention, a lower triangular mask is applied so that each position can only attend to itself and previous positions — preventing the model from “seeing the future.” **Multi-head attention** runs h attention functions in parallel, each in a lower-dimensional subspace of size $d_k = d_{model}/h$, allowing the model to capture different types of relationships simultaneously. The outputs of all heads are concatenated and linearly projected back to d_{model} .

- First, implement a module for masked scaled dot-product attention.
- Then, use it to build a masked multi-head attention module.
- The mask must be a **causal mask** (lower triangular matrix).

Your implementation must perform the following steps:

- Compute Query (Q), Key (K), and Value (V) projections from the input.
- Split the embeddings into multiple heads.
- Apply masked scaled dot-product attention independently to each head.
- Ensure future tokens are masked before applying softmax.
- Concatenate outputs from all heads.
- Apply a final linear projection.

Use the following configuration:

$$d_{model} = 8, \quad h = 2, \quad \text{sequence length} = 4$$

Generate input:

```
torch.manual_seed(42)
X = torch.randn(1, 4, 8)
```

Print the following:

- Projected Query, Key, Value matrices (before and after splitting heads)
- Mask matrix
- Masked attention scores
- Attention weights
- Final output

Also verify programmatically that:

- All future positions have zero attention weight using assertions.

5. In scaled dot-product attention, the attention scores are divided by $\sqrt{d_k}$ before applying softmax.

In this question, you will **empirically visualize the effect of scaling** on attention distributions.

- Generate random Query (Q) and Key (K) matrices with varying dimensionality d_k .
- Fix the random seed using `torch.manual_seed(42)` inside the visualization function for reproducibility.

- Compute attention scores in two ways:
 - Without scaling: QK^T
 - With scaling: $\frac{QK^T}{\sqrt{d_k}}$
- Apply softmax to obtain attention weights.
- Visualize and compare the distributions.

Use the following setup:

- Sequence length = 4
- Try different values of d_k : 4, 16, 64

Plot:

- Attention weight matrices (heatmaps) with annotated values
- Histogram of attention weights with axis labels

Based on your observations, comment on:

- The effect of increasing d_k without scaling on the attention distribution
- Why dividing by $\sqrt{d_k}$ stabilizes training

6. Interpreting CNN Decisions using Grad-CAM

Setup Instructions (PyTorch Grad-CAM)

We will use the `pytorch-grad-cam` library.

- Install the package:


```
pip install grad-cam
```
- Verify installation:


```
from pytorch_grad_cam import GradCAM
```

Consider a convolutional neural network (CNN) trained for MNIST digit classification. The architecture is given below:

- Conv(1, 16, kernel=3) + ReLU
- MaxPool(2)
- Conv(16, 32, kernel=3) + ReLU
- MaxPool(2)
- Flatten
- FC(128) + ReLU
- FC(10) (logits)

Let $f(x) \in \mathbb{R}^{10}$ denote the output logits for input image x , and let y^c denote the logit corresponding to class c .

1. **(Implementation)** Implement Grad-CAM for the above network using PyTorch. Use the last convolutional layer as the target layer.
 2. **(Visualization)** For a set of correctly classified MNIST test images:
 - Generate Grad-CAM heatmaps.
 - Overlay the heatmap on the original image.
 3. **(Analysis)** Based on your visualizations:
 - Identify which regions of the digit contribute most to classification.
 - Compare heatmaps across different digits (e.g., 1 vs 8).
 - Comment on whether the model focuses on semantically meaningful regions.
 4. **(Failure Cases)** Repeat the above for misclassified examples and discuss:
 - Whether Grad-CAM highlights misleading regions.
 - Possible reasons for failure (e.g., ambiguous strokes, noise).
 5. **(Extension)** Modify the input image (e.g., occlude parts of the digit) and observe how the Grad-CAM map changes. Relate this to model robustness.
7. Consider a simple Recurrent Neural Network (RNN) defined for time steps $t = 1, 2, 3$:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$o_t = W_{hy}h_t + b_y$$

$$\hat{y}_t = o_t$$

The total loss is given by

$$L = \sum_{t=1}^3 L_t, \quad \text{where} \quad L_t = \frac{1}{2} \|\hat{y}_t - y_t\|^2$$

Answer the following:

1. Derive the expression for $\frac{\partial L}{\partial W_{hy}}$.
2. Let

$$a_t = W_{hh}h_{t-1} + W_{xh}x_t + b_h$$

Derive the recursive expression for

$$\delta_t = \frac{\partial L}{\partial a_t}.$$

3. Using the result above, derive $\frac{\partial L}{\partial W_{hh}}$.
4. Briefly explain why vanishing and exploding gradients occur in Backpropagation Through Time (BPTT).

8. (Programming Challenge: Build a VGG like Network from Scratch)

In this problem, you will design and train a smaller version of the VGG architecture for image classification.

- **Dataset:** Fashion-MNIST (`torchvision.datasets.FashionMNIST`)

Task: Classify images into 10 clothing categories.

Architecture Requirements: Design a network with the following principles:

- Use stacks of convolutional layers with small filters (3×3)
- Apply ReLU activation after each convolution
- Use MaxPooling layers to reduce spatial dimensions
- Use at least 3 convolutional blocks
- Add fully connected layers at the end for classification

Example Structure (you may modify):

- Block 1: Conv $\rightarrow$ Conv $\rightarrow$ MaxPool
- Block 2: Conv $\rightarrow$ Conv $\rightarrow$ MaxPool
- Block 3: Conv $\rightarrow$ Conv $\rightarrow$ MaxPool
- Fully Connected Layers $\rightarrow$ Output (Softmax)

Training Setup:

- Optimizer: Adam
- Learning rate: 0.001
- Train for at least 15 epochs
- Use Cross-Entropy Loss

Requirements:

- Implement the architecture with pytorch (do NOT use pretrained models)
- Use Batch Normalization and/or Dropout
- Track training and validation accuracy

Report:

- Plot training and validation accuracy and loss curves for all epochs

- Report final training and test accuracy
- Print total number of trainable parameters in your model
- Report training time per epoch